

Junior Makers · Class 24

Processing: Interactive Drawing Machine

Code your own painting app

Coordinates

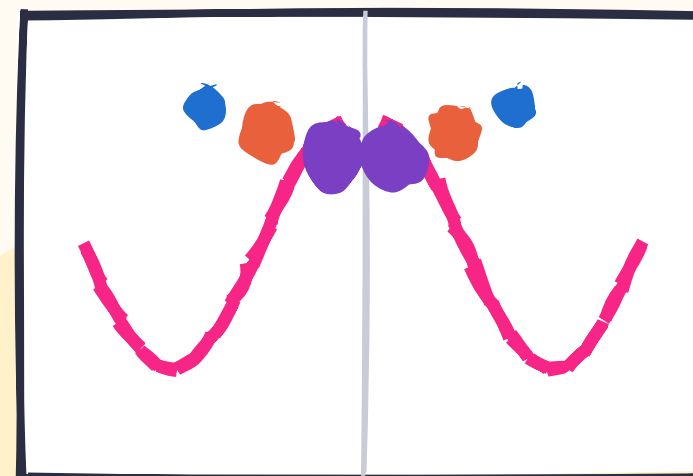
draw() loop

Variables

Symmetry art

Did you know?

Processing was made in 2001 by two artists at MIT so that artists and designers could learn to code. Millions now use it.





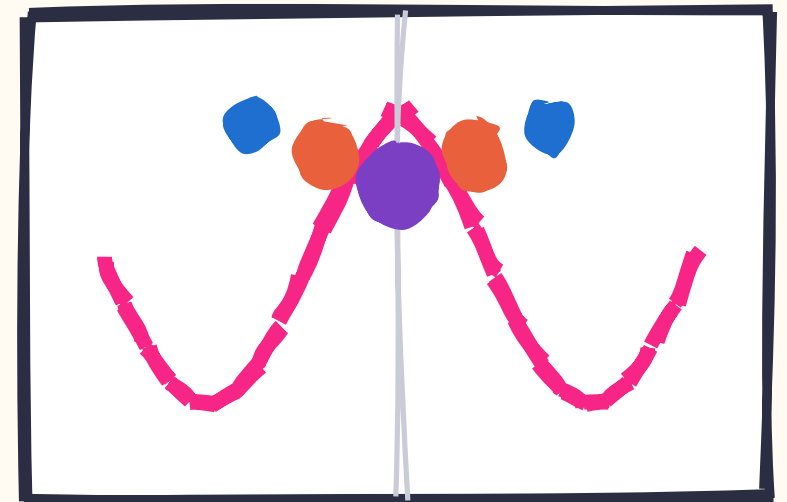
Let's think...

Could you build your own paint app?



Paint apps are code

Apps like MS Paint or drawing apps on tablets seem complicated. But the heart of a paint app is just a few lines: **wherever the mouse is, draw a shape**. Today you'll build your own, then add tricks no normal paint app has!



Too hard for us?

Only experts can?

A few lines of code!

Today: you code a **drawing machine** in Processing: brush, colours, speed-sensitive lines and magic **mirror symmetry**.





Learning objectives

By the end of this class I can...



1  **Coordinates:** place shapes on a screen using **x, y** coordinates

Proof: I drew a face from ellipses and rectangles

2  **draw() loop:** explain **setup()** and **draw()** and use **mouseX, mouseY**

Proof: my brush follows the mouse

3  **Variables & keys:** change colour and size with **variables**, keys and mouse speed

Proof: pressing keys changes my brush

4  **Create & share:** make a **symmetry drawing machine** and save my art

Proof: a saved PNG of my artwork





Objective 1 · Coordinates · video

Video time!



What is Creative Coding?

Yeti Learn · about 4 minutes · youtu.be/eJOObl-Zd2A

Watch for...

- 1 What is **creative coding**?
- 2 Name two things people make with creative code.
- 3 Why would an artist use code instead of a brush?



After the video: If you could make a painting app with ONE magic feature, what would it be?



Objective 1 · Coordinates

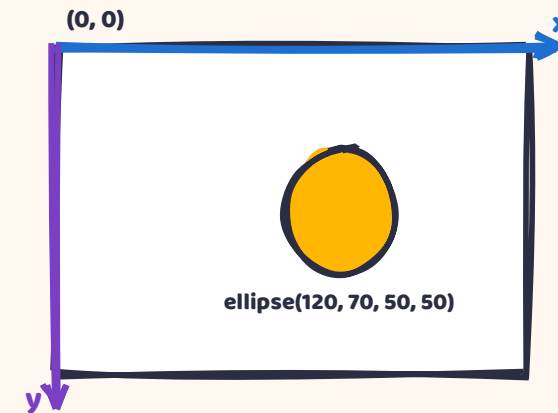
Every pixel has an address

The idea

The window is a grid of pixels. **(0, 0)** is the **top-left** corner. **x** grows to the right, **y** grows **downwards**.

- `ellipse(x, y, width, height)`: circle or oval
- `rect(x, y, width, height)`: rectangle
- `line(x1, y1, x2, y2)`: a line
- `fill(r, g, b)`: colour inside

Example: a map with a grid



City maps have letters across the top and numbers down the side. 'Shop at C4' tells you exactly where to look. Processing finds every pixel the same way.



Class game

Human pixels



How to play

1. Put a big grid on the board: x across the top (0 to 10), y down the side (0 to 10).
2. The 'programmer' gives commands: **ellipse at (3, 4), rect at (6, 2), 3 wide, 2 tall.**
3. The 'computer' (a student) draws exactly what is said.
4. Goal: draw a face using only commands. No pointing!

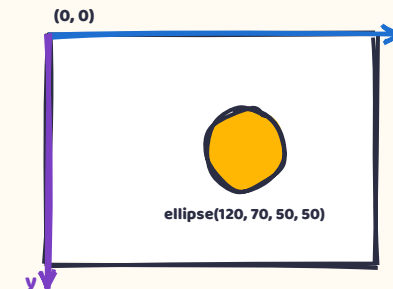
Then discuss

What went wrong? Did anyone forget that y goes DOWN?



Big idea

Computers need exact coordinates. Being exact is a superpower in coding.



Objective 1 · Coordinates · Build step 1 of 4

Draw a face with code



```
void setup() {  
  size(600, 400);           // width, height  
  background(255, 240, 200); // light yellow  
}  
  
void draw() {  
  fill(255, 210, 160);  
  ellipse(300, 200, 220, 240); // head  
  fill(40);  
  ellipse(260, 170, 25, 25);   // left eye  
  ellipse(340, 170, 25, 25);  // right eye  
  noFill();  
  arc(300, 230, 100, 60, 0, PI); // smile  
}
```

Processing 4

Download free from processing.org. Type the code, press the round **Run** button.

Experiment

Change one number at a time. Which number moves the eyes apart? Which makes the head wider?

Colours

fill(red, green, blue): each 0–255. Like NeoPixels in JM-21!



Objective 2 · draw() loop

setup() once, draw() forever



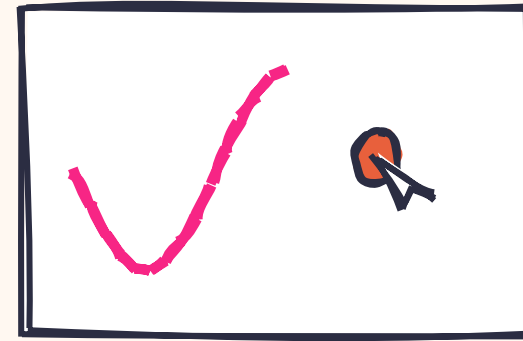
The idea

setup() runs **once** at the start: set the window size and background.

draw() runs again and again, about **60 times a second**, like a forever loop.

Inside draw(), **mouseX** and **mouseY** always hold where the mouse is *right now*. Draw at (mouseX, mouseY) and the shape follows the mouse!

Example: a flipbook again



Each time draw() runs is one page of a flipbook. If we don't clear the background, every page keeps the old drawing, so the shapes pile up into a painting!





Objective 2 · draw() loop · Build step 2 of 4

A brush that follows the mouse



```
void setup() {  
  size(800, 600);  
  background(255);           // white, only ONCE  
}  
  
void draw() {  
  if (mousePressed) {       // only when the button is held  
    noStroke();  
    fill(255, 0, 120, 60); // last number = see-through  
    ellipse(mouseX, mouseY, 30, 30);  
  }  
}
```

Why background in setup?

Move **background(255)** into draw() and see what happens. (The painting disappears every frame!)

See-through

The 4th number in fill() is **alpha**: 0 = invisible, 255 = solid. Low alpha makes soft, watercolour-like layers.

Check

Hold the mouse button and draw: pink soft blobs!





Objective 3 · Variables & keys

Variables remember things



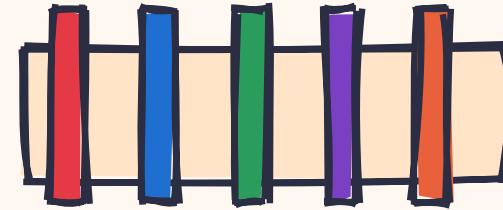
The idea

A **variable** is a named box that holds a value. `float size = 30;`

We can change it while the program runs:

- **keyPressed()**: when a key is pressed, change colour or size
- **mouse speed**: `dist(mouseX, mouseY, pmouseX, pmouseY)` tells how far the mouse moved since the last frame. Fast = thick line!

Example: a pencil case



Your pencil case remembers which pens you have. When you swap a blue pen for a red one, every drawing after that is red, until you swap again. A colour variable works the same way.



Objective 3 • Variables & keys • Build step 3 of 4

The rainbow speed brush

```
float hue = 0;

void setup() {
  size(800, 600);
  colorMode(HSB, 360, 100, 100, 100); // rainbow-friendly colours
  background(0, 0, 100);
}

void draw() {
  if (mousePressed) {
    float speed = dist(mouseX, mouseY, pmouseX, pmouseY);
    strokeWeight(map(speed, 0, 50, 2, 20)); // fast = thick
    stroke(hue, 80, 90, 80);
    line(pmouseX, pmouseY, mouseX, mouseY);
    hue = (hue + 1) % 360; // slowly change colour
  }
}

void keyPressed() {
  if (key == 'c') background(0, 0, 100); // clear
  if (key == 's') saveFrame("my-art-###.png"); // save!
}
```

HSB colour

Hue 0–360 goes round the rainbow: 0 red, 120 green, 240 blue. Adding 1 each frame slides through every colour.

pmouseX

Where the mouse was last frame. A line from there to here makes smooth strokes, even when you move fast.

Keys

Press **c** to clear and **s** to save a picture in your sketch folder.



Objective 4 · Create & share

Mirror magic: symmetry



The idea

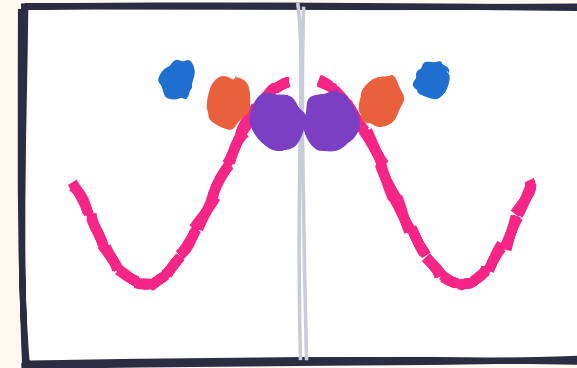
To mirror a point left-right, keep y the same and flip x : $\text{width} - x$.

Draw the same line twice:

```
line(pmouseX, pmouseY, mouseX, mouseY);  
line(width - pmouseX, pmouseY, width - mouseX,  
mouseY);
```

Add top-bottom too ($\text{height} - y$) for 4-way kaleidoscope art!

Example: a butterfly painting



Fold paper in half, paint on one side, press it closed and open it: a perfect butterfly! Our code does the 'fold' with maths, live, as you draw.



Objective 4 · Create & share · Build step 4 of 4

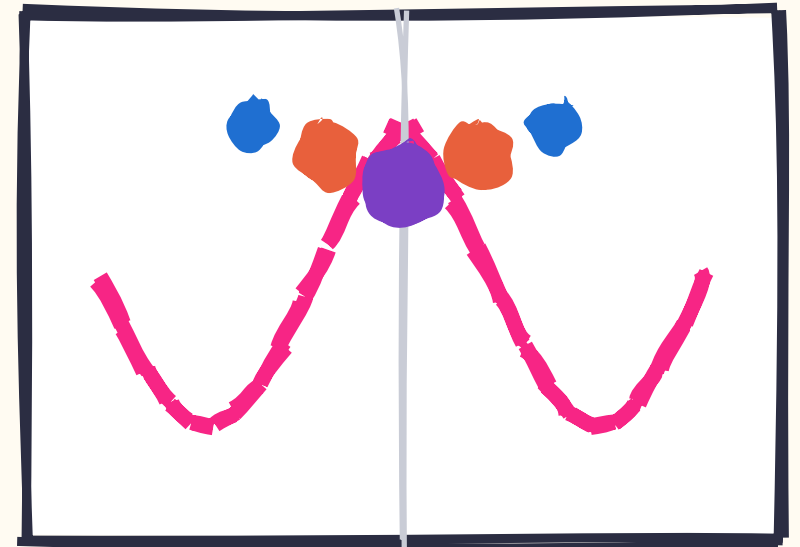
Add symmetry

```
// inside draw(), after your first line(...)
line(width - pmouseX, pmouseY, width - mouseX, mouseY); // left-right

// 4-way kaleidoscope: add these too
line(pmouseX, height - pmouseY, mouseX, height - mouseY);
line(width - pmouseX, height - pmouseY,
      width - mouseX, height - mouseY);
```

Try it

Draw slowly near the middle, then fast near the edges. Draw a single wavy line: what appears?



Your twist

Add a key that turns symmetry on and off (a true/false variable).



Real world

Creative coding in the real world



Digital artists

Exhibit code-made art in galleries



Game makers

Draw every frame of a game with code



Data artists

Turn weather or music data into pictures



Designers

Generate patterns for clothes, posters and logos



Which would you rather make: art with a brush, or art with code? Why?

Islamic patterns

Many traditional geometric patterns can be generated with loops and symmetry, just like ours.



Objective 4 · Create & share · share

Digital art gallery

How it works

Save your 2 best pictures (press **s**). Put them in the shared class folder. The teacher shows them as a slideshow.

For each artwork, the artist says one thing their **code** does that a normal paint app can't.

Artist card

My brush trick

Colour idea

Symmetry: 2 or 4 way?

A bug I fixed



Test like an engineer

Bug hunt



Bug	Likely cause	Did it happen?
Red error: missing ;	every line needs a semicolon	
Drawing disappears	background() inside draw()	
Nothing when clicking	if (mousePressed) misspelt	
Mirror on wrong side	width – x vs height – y mixed up	
Colours all grey	colorMode HSB missing	

Debug checklist

Processing highlights the line with the error in yellow. Read the message at the bottom.



Use `println(mouseX, mouseY);` to see the numbers in the console.



Level up!

Artist challenges

Choose your challenge



Stamp tool: press 1 for circles, 2 for squares, 3 for stars.



6-way kaleidoscope: use rotate() in a loop of 6 to draw like a snowflake.



Colour picker: click a palette strip at the top to choose colours.



Sound brush: use the Sound library: louder = bigger brush.

Think like an engineer

How would you make an 'undo' button?
What would the program need to remember?

Boss level

Make a 'spirograph' that draws by itself using sin() and cos().



Think about it

Blocks vs typed code



Blocks (Scratch, mBlock)

- ✓ No spelling mistakes
- ✓ Easy to see the structure
- ✗ Slower for big programs

Typed code (Processing, Python)

- ✓ Fast once you know it
- ✓ Used by professionals
- ✗ One missing ; breaks it



You can do both!

Today you wrote real Java-based code. The ideas (loops, variables, if) are the same as in blocks.





Objective check

Can I really do it? Prove it.



01 Where is (0, 0)? Show the line of code that draws your face's left eye.



02 What is the difference between setup() and draw()?



03 Show a key that changes your brush. Which variable does it change?



04 Show your saved artwork. Explain how width – x makes the mirror.



Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Gallery slideshow on the projector: each artist presents one piece in 30 seconds.





Show what you know

What did we learn?

- ☒ (0, 0) is top-left; y grows downwards
- ☒ setup() runs once, draw() runs about 60 times a second
- ☒ mouseX and mouseY tell you where the mouse is
- ☒ Variables change your brush while the program runs
- ☒ width – x mirrors a drawing left to right

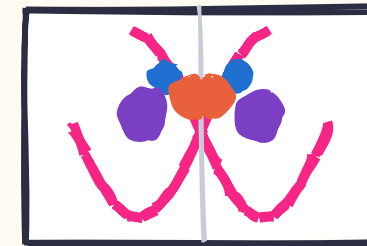


Exit ticket: your drawing vanishes every frame. Which line is in the wrong place?



At home

Find symmetry at home: a leaf, a face, a carpet. Is it 2-way or 4-way?



Next class:
Smart Traffic
Junction with Car
Counter!

